VOLUME 5 OF 12

IT STUDY HUB

COMPLETE JAVA PROGRAMMING SERIES

MODULE 5

OBJECT-ORIENTED PROGRAMMING

Authored by:

Mithila Rani Saha & Dithsa Dutta

www.itstudyhub.dpdns.org

Welcome to Module 5

About This Module

Welcome to Volume 5 of the IT Study Hub Complete Java Programming Series. In Modules 1 through 4, we wrote code in a "Procedural" style—a straight, top-to-bottom list of commands. While procedural programming works for small scripts, it completely falls apart when building massive applications like banking systems or 3D video games.

To build real-world software, we must model real-world concepts. This is **Object-Oriented Programming (OOP)**. Instead of just writing loose methods and scattered variables, you will learn to bundle data and behaviors together into "Objects". This is the absolute core of Java.

Learning Outcomes

- Understand the paradigm shift from Procedural to Object-Oriented thinking.
- Design custom `Classes` and instantiate `Objects` from them in memory.
- Initialize object states using Default and Parameterized `Constructors`.
- Protect sensitive data using the `private` access modifier and `Encapsulation`.
- Reuse code efficiently by applying `Inheritance` and the `super` keyword.
- Implement `Polymorphism` through Method Overloading and Method Overriding.
- Define strict blueprints for your code using `Abstraction` via Abstract Classes and Interfaces.

Module Coverage

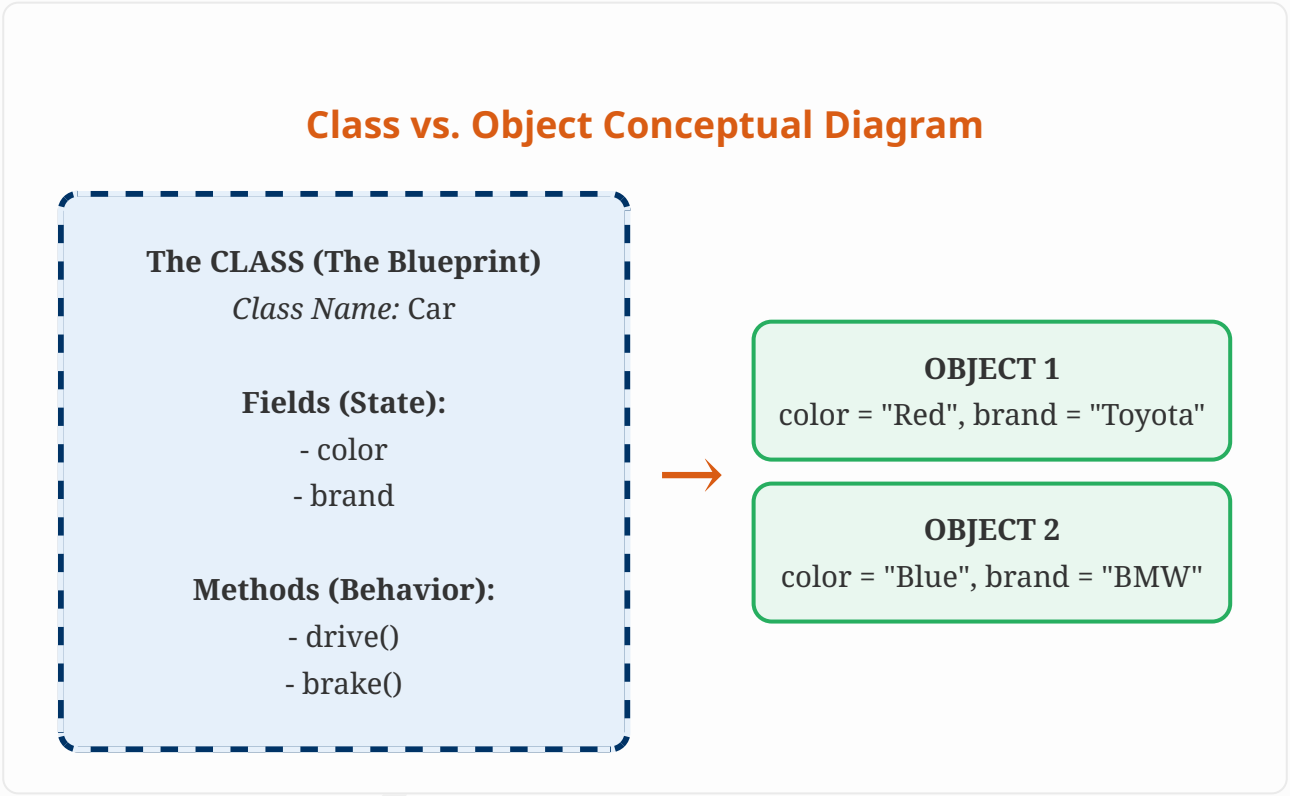
We begin by defining Classes and Objects using familiar analogies (Cars, Students, Bank Accounts). We then explore the four pillars of OOP: Encapsulation,

Inheritance, Polymorphism, and Abstraction. We will demystify crucial keywords like `this`, `super`, and `final`, and map out Access Modifiers. We conclude with an extensive "Bank Account Management System" project and our standard 45+ question assessment section.

IT STUDY HUB

1. Introduction to OOP, Classes, and Objects

Object-Oriented Programming is a methodology that designs software around data (objects) rather than just logic (functions). An object is a self-contained entity that has state (data/fields) and behavior (actions/methods).



Concept: Classes, Objects, Fields, and Methods

A Class is a blueprint. An Object is an actual, physical instance created from that blueprint. Fields are variables inside the class. Methods are functions inside the class.

> ***Real-Life Analogy:***
> *If a Class is the architectural blueprint for a house, an Object is the actual house built from that blueprint. You can't live in a blueprint! You use one blueprint to build many distinct houses on a street. Every house has fields (door color, number of rooms) and methods (open door, turn on lights).*

> **Simple Explanation:**
> When you define a Class, no memory is consumed for data. It's just a definition. When you use the `new` keyword, Java takes that blueprint and constructs a physical Object in the computer's memory.

Syntax / Structure:

```
// Defining the Class
public class ClassName {
    // Fields (State)
    dataType variableName;

    // Methods (Behavior)
    public void methodName() { ... }
}

// Creating an Object in main()
ClassName objectName = new ClassName();
```

Example Code:

```
// The Blueprint
class Student {
    // Fields (State)
    String name;
    int age;

    // Method (Behavior)
    public void study() {
```

```
        System.out.println(name + " is studying hard!");
    }
}

public class MainApp {
    public static void main(String[] args) {
        // Creating Object 1
        Student s1 = new Student();
        s1.name = "Alice";
        s1.age = 20;

        // Creating Object 2
        Student s2 = new Student();
        s2.name = "Bob";
        s2.age = 22;

        // Calling their behaviors
        s1.study();
        s2.study();
    }
}
```

```
Expected Output:
Alice is studying hard!
Bob is studying hard!
```

Code Explanation:
We created a `Student` blueprint with a name, an age, and an action (`study`). In the `main` method, we used the `new` keyword twice to spawn two separate `Student` objects in memory. Changing Alice's age does not affect Bob's age because they are distinct objects.

> ***Real-world Use:***
> *In an online shopping app, `Product` is a class. A specific pair of shoes is an object. A specific laptop is another object.*

> **Common Mistakes:**
>
> - Trying to assign a value directly to a blueprint without creating an object first (e.g., `Student.name = "Alice";` — this throws a compiler error because blueprints can't hold data).

Best Practices:

- Class names should always start with a Capital Letter (PascalCase). Fields and methods should start with a lowercase letter (camelCase).

Practice Task:
Create a `Dog` class with fields `name` and `breed`, and a `bark()` method. In main, create two Dog objects and make them bark.

Class vs Object Comparison

Aspect	Class	Object
Definition	A logical blueprint or template.	A physical reality created from the class.
Memory	Does not consume RAM when defined.	Consumes RAM when instantiated via new.
Existence	Declared once per concept.	Can be instantiated thousands of times.
Example	`class Employee {{ ... }}`	`Employee john = new Employee();`

IT STUDY HUB

2. Constructors and the 'this' Keyword

IT STUDY HUB

Concept: Constructors (Default & Parameterized)

A special block of code inside a class used strictly to initialize objects immediately when they are created.

> ***Real-Life Analogy:***
> *When you buy a brand new smartphone, it boots up and runs a one-time setup wizard to configure your language and time zone. A Constructor is the 'setup wizard' for a Java object.*

> **Simple Explanation:**
> A constructor looks like a method, but it has TWO strict rules: 1. It MUST have the exact same name as the Class. 2. It has NO return type (not even void). It runs automatically the moment you use the `new` keyword.

Syntax / Structure:

```
class Car {
    // Constructor
    public Car(parameters) {
        // initialization code
    }
}
```

Example Code:

```
class Car {
    String model;
    int year;

    // 1. Default Constructor (No parameters)
    public Car() {
        model = "Unknown";
        year = 2000;
        System.out.println("Default car built!");
    }

    // 2. Parameterized Constructor
    public Car(String m, int y) {
        model = m;
```

```
        year = y;
        System.out.println("Custom car built!");
    }
}

public class Dealership {
    public static void main(String[] args) {
        Car c1 = new Car();             // Triggers Default
        Car c2 = new Car("Honda", 2024); // Triggers Parameterized

        System.out.println(c1.model + " " + c1.year);
        System.out.println(c2.model + " " + c2.year);
    }
}
```

```
Expected Output:
Default car built!
Custom car built!
Unknown 2000
Honda 2024
```

Code Explanation:
We have two constructors (Constructor Overloading). `new Car()` looks for the constructor with no inputs. `new Car("Honda", 2024)` looks for the constructor accepting a String and an int, setting up the object's data instantly.

> ***Real-world Use:***
> *Connecting to a database. The moment a `DatabaseConnection` object is created, its constructor automatically feeds in the username and password to secure the connection immediately.*

> **Common Mistakes:**
> - Putting `void` in front of a constructor (e.g., `public void Car()`). Java will treat it as a regular method and the setup logic will fail to run automatically on object creation.

Best Practices:

- If you write a parameterized constructor, Java removes the hidden invisible default constructor. It's a best practice to always write an empty default constructor manually just in case.

Practice Task:
Create a `Book` class with a constructor that takes the `title` and `author`. Create a Book object in main.

IT STUDY HUB

Concept: The 'this' Keyword

A reference variable that refers to the CURRENT object.

> ***Real-Life Analogy:***
> *If you say 'My leg hurts', 'My' refers to yourself. In Java, if a Car object says `this.color`, it means 'MY specific color', not the color of any other car.*

> **Simple Explanation:**
> When the names of a constructor's parameters are identical to the Class's field names, Java gets confused. It suffers from 'Variable Shadowing'. We use `this.` to tell Java, 'Hey, I am talking about the Class's field, not the parameter.'

Syntax / Structure:

```
class Employee {
    String name;

    public Employee(String name) {
        this.name = name; // this.name = Class Field | name = Parameter
    }
}
```

Example Code:

```
class Employee {
    String name;
    double salary;

    public Employee(String name, double salary) {
        // Without 'this', name = name just assigns the parameter to itself!
        this.name = name;
        this.salary = salary;
    }

    public void printDetails() {
        System.out.println("Employee: " + this.name + " earns $" + this.salary);
    }
}
```

```
public class Office {
    public static void main(String[] args) {
        Employee emp = new Employee("Sarah", 75000);
        emp.printDetails();
    }
}
```

```
Expected Output:
Employee: Sarah earns $75000.0
```

Code Explanation:
In the constructor, `this.name` points to the blueprint's empty storage box, while `name` (without `this`) points to the temporary data passed in (Sarah). We successfully place 'Sarah' into the permanent storage box.

Real-world Use:
Used universally in enterprise Java (Spring, Android) to safely map incoming JSON/API data to internal object models without naming conflicts.

Common Mistakes:

- Writing `name = this.name;`. This takes the empty null value from the class and overwrites the good data you just passed in! Always assign TO `this.field`.

Best Practices:

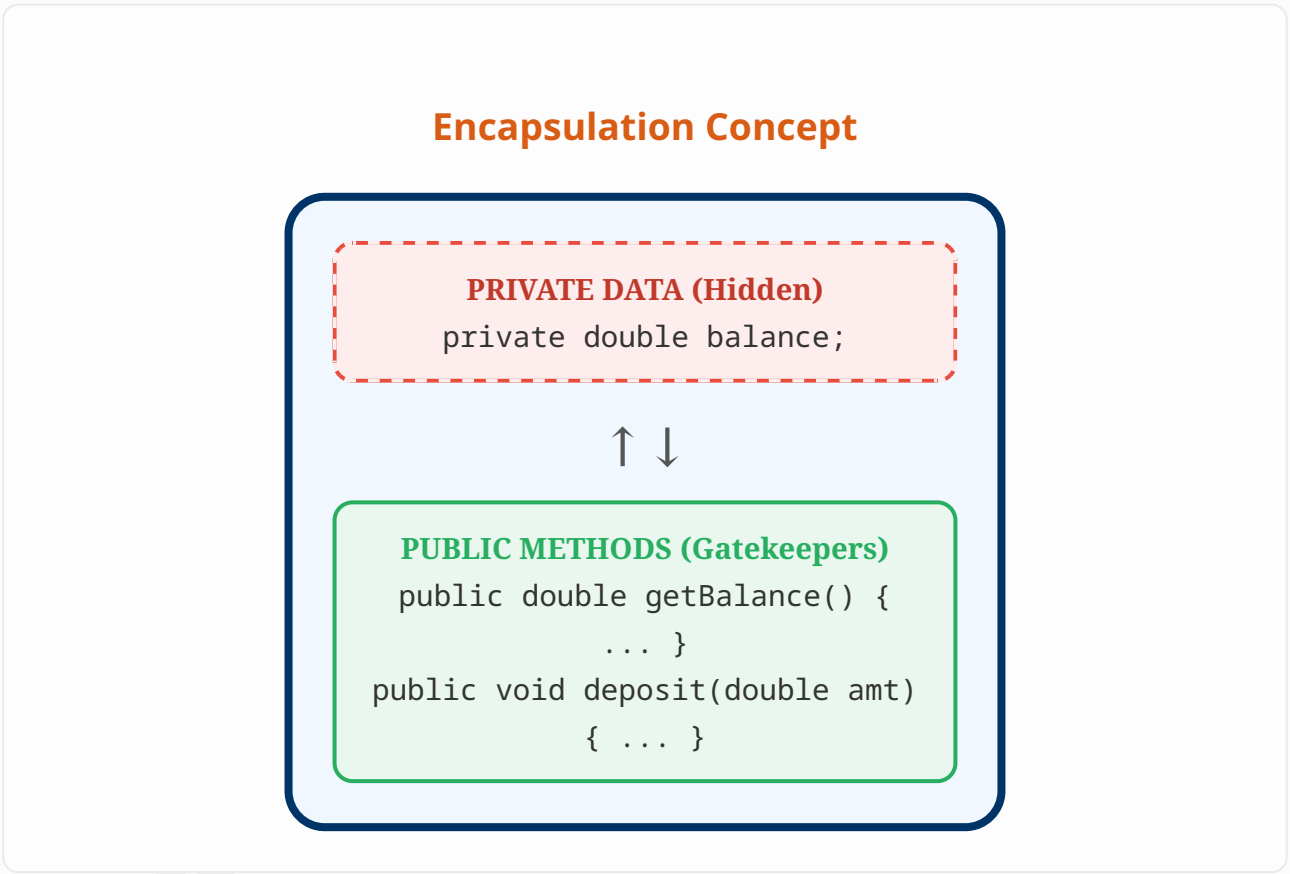
- Always name your constructor parameters exactly the same as your class fields and use `this.` to assign them. It makes code highly readable.

Practice Task:
Create a `Point` class with x and y fields. Use `this` in a constructor to set their values.

---

3. Pillar 1: Encapsulation

Encapsulation is the mechanism of hiding the sensitive data of a class from the outside world, and only providing access to it through tightly controlled public methods (Getters and Setters).



Concept: Private Fields and Getters/Setters

Securing object state by making fields `private` and methods `public`.

> ***Real-Life Analogy:***
> *A Bank ATM. The bank vault (private data) is full of cash. You cannot directly walk up and grab the cash (direct field access). You must use the ATM machine (public methods) to verify your PIN and withdraw safely.*

> **Simple Explanation:**
> If you leave fields `public`, any other class can change them to invalid values (like setting age to -500). By making them `private`, other classes get blocked. We write public 'Getter' methods to let others read the data safely, and 'Setter' methods to let them change it only if it passes our validation rules.

Syntax / Structure:

```
private dataType fieldName;

public dataType getFieldName() { return fieldName; }
public void setFieldName(dataType val) { fieldName = val; }
```

Example Code:

```
class BankAccount {
    // 1. Private data (Cannot be touched directly)
    private double balance = 1000;

    // 2. Getter (Allows reading)
    public double getBalance() {
        return balance;
    }

    // 3. Setter (Allows modification with validation)
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount; // Safe update
            System.out.println("Deposited: $" + amount);
        } else {
            System.out.println("Error: Deposit must be positive!");
```

```
        }
    }
}

public class BankApp {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount();

        // acc.balance = 5000000; // ERROR! Balance is private!

        acc.deposit(500); // Using the public method
        System.out.println("Current Balance: $" + acc.getBalance());
    }
}
```

```
Expected Output:
Deposited: $500.0
Current Balance: $1500.0
```

Code Explanation:
We successfully protected the bank account. If a hacker tries `acc.balance = 500000`, the compiler immediately blocks it. The only way to interact with the money is through the `deposit()` method, which verifies the amount is positive.

Real-world Use:
Ensuring a User object's `password` field is private, and the setter automatically encrypts the password before saving it to memory.

Common Mistakes:

- Making fields private but forgetting to write getters, effectively locking the data so firmly that even your own program can't read it!

Best Practices:

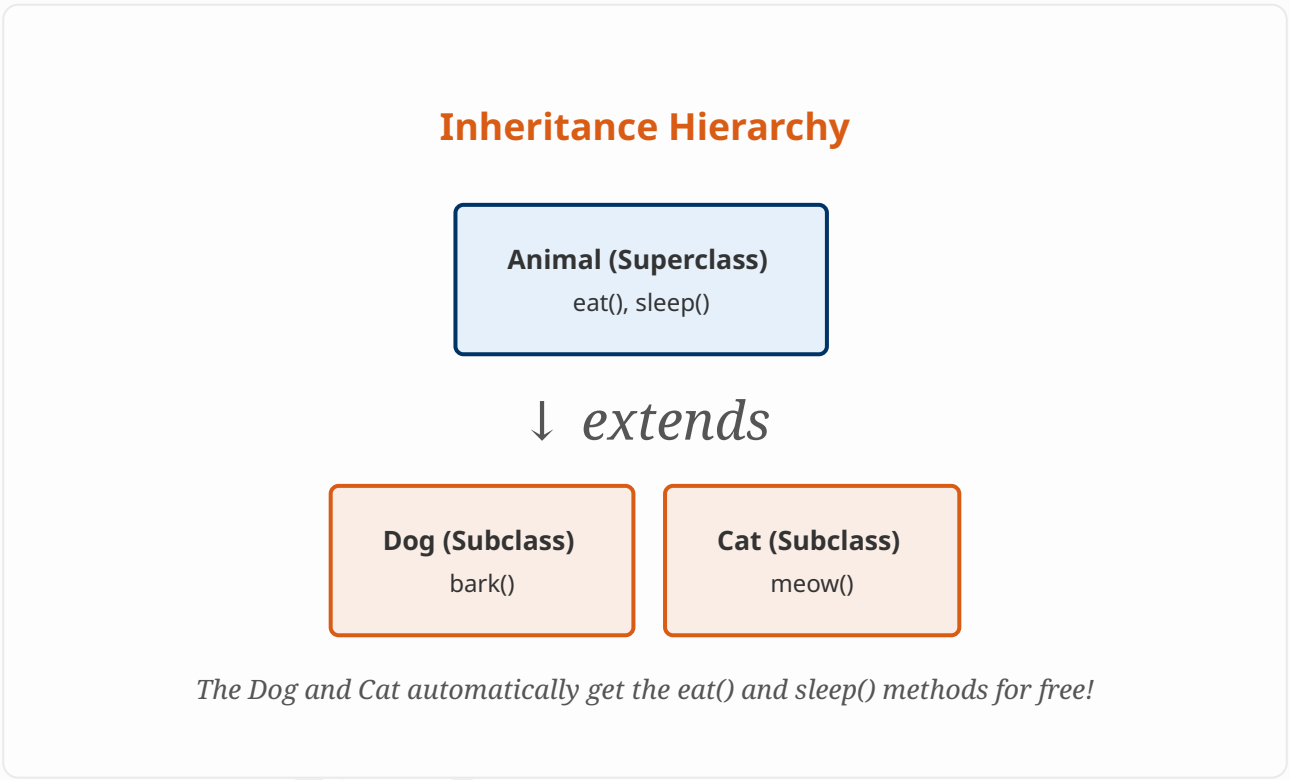
- Make ALL fields in your classes `private` by default. Generate getters and setters only for the data that absolutely needs to be accessed outside the class.

Practice Task:
Create a `Person` class with a private `age` field. Write a setter that only updates the age if the input is between 0 and 120.

IT STUDY HUB

4. Pillar 2: Inheritance

Inheritance is a mechanism where a new class (Subclass/Child) derives properties and methods from an existing class (Superclass/Parent). It promotes immense code reusability.



The Dog and Cat automatically get the eat() and sleep() methods for free!

Concept: The 'extends' Keyword and 'super'

Using `extends` to inherit logic. Using `super` to communicate with the parent class.

> ***Real-Life Analogy:***
> *A Parent has a house and a car. A Child inherently gets access to the parent's house and car. However, if the child wants to buy a specific toy, they ask the parent to handle the money using the `super` command.*

> **Simple Explanation:**
> Instead of copying and pasting the `eat()` method into 50 different animal classes, we put it in the `Animal` class. Then `Dog extends Animal`. The Dog gets all non-private fields and methods from Animal automatically. The `super()` command is used in the child's constructor to trigger the parent's constructor.

Syntax / Structure:

```
class ChildClass extends ParentClass {
    public ChildClass() {
        super(); // Calls parent constructor
    }
}
```

Example Code:

```
// Parent Class
class Employee {
    String name;

    public Employee(String name) {
        this.name = name;
    }

    public void work() {
        System.out.println(name + " is working.");
    }
}

// Child Class inherits from Employee
class Manager extends Employee {
```

```
    int teamSize;

    public Manager(String name, int teamSize) {
        super(name); // Sends name up to Employee's constructor!
        this.teamSize = teamSize;
    }

    public void conductMeeting() {
        System.out.println(name + " is leading a meeting for " + teamSize + " people.");
    }
}

public class Company {
    public static void main(String[] args) {
        Manager mgr = new Manager("Diana", 10);
        mgr.work(); // Inherited method!
        mgr.conductMeeting(); // Manager-specific method!
    }
}
```

```
Expected Output:
Diana is working.
Diana is leading a meeting for 10 people.
```

Code Explanation:
Notice how `Manager` never explicitly defined a `work()` method. It absorbed it from `Employee`. In the `Manager` constructor, `super(name)` is critical: it passes 'Diana' up to the Parent so the Parent can do the job of assigning it to the `name` field.

> ***Real-world Use:***
> *In Android development, you create screens by writing `class MainActivity extends AppCompatActivity`. You instantly inherit thousands of lines of Google's UI code for free.*

> **Common Mistakes:**
> - Trying to inherit from multiple parent classes (e.g., `class Dog extends Animal, Pet`). Java strictly prohibits 'Multiple Inheritance' with classes to avoid logical conflicts.

Best Practices:

- The `super()` call MUST be the absolute first line of code inside a child constructor. Java enforces this to ensure the parent is fully built before the child is built.

Practice Task:
Create a `Vehicle` class with a `startEngine()` method. Create a `Motorcycle` class that extends `Vehicle`. Call `startEngine()` from a Motorcycle object.

IT STUDY HUB

5. Pillar 3: Polymorphism

Polymorphism means "many forms". It allows us to perform a single action in different ways. In Java, this is primarily achieved through Method Overloading (Compile-time) and Method Overriding (Run-time).

IT STUDY HUB

Concept: Method Overriding (@Override)

When a Child class provides a specific, custom implementation for a method that is already provided by its Parent class.

> ***Real-Life Analogy:***
> *Every vehicle has a 'start engine' process. In the Parent `Vehicle` class, `start()` might just say 'Vroom'. But an ElectricCar is a Vehicle too. It needs to override that method to instead say 'Silent startup' because its engine works differently.*

> **Simple Explanation:**
> If a parent method doesn't quite fit the child's needs, the child can rewrite it. You write the exact same method signature (name and parameters) in the child class. We use the `@Override` annotation above the method to tell the compiler 'I am intentionally replacing the parent's version'.

Syntax / Structure:

```
class Parent {
    public void speak() { System.out.println("Hello"); }
}

class Child extends Parent {
    @Override
    public void speak() { System.out.println("What's up?"); }
}
```

Example Code:

```
class Animal {
    public void makeSound() {
        System.out.println("Some generic animal sound...");
    }
}

class Dog extends Animal {
    @Override // Annotation guarantees we spelled it right!
    public void makeSound() {
        System.out.println("Bark! Bark!");
    }
```

```
}

class Cat extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Meow~");
    }
}

public class Zoo {
    public static void main(String[] args) {
        Animal generic = new Animal();
        Dog buddy = new Dog();
        Cat luna = new Cat();

        generic.makeSound();
        buddy.makeSound(); // Triggers overridden version
        luna.makeSound();  // Triggers overridden version
    }
}
```

```
Expected Output:
Some generic animal sound...
Bark! Bark!
Meow~
```

Code Explanation:
All three objects call the identically named `makeSound()` method. However, because `Dog` and `Cat` successfully *overrode* the method, Java executes their specific custom logic instead of the parent's generic logic.

> ***Real-world Use:***
> *Overriding the default Java `toString()` method in your custom classes so that when you print the object, it prints formatted readable text instead of a random memory hash.*

> **Common Mistakes:**
>
> - Changing the parameters in the child method. If you change the parameters, you are OVERLOADING, not OVERRIDING. The parent method will not be replaced!

Best Practices:

- Always type `@Override` above your method. It forces the compiler to double-check your work. If you misspell `makeSound()` as `mkeSound()`, the compiler will throw an error immediately.

Practice Task:
Create a `Shape` class with `draw()`. Create a `Circle` class that extends `Shape` and overrides `draw()` to print 'Drawing a round circle'.

Overloading vs. Overriding

Feature	Method Overloading (Compile-Time)	Method Overriding (Run-Time)
Where?	Inside the SAME class.	Between a PARENT and CHILD class.
Method Name	Must be the exact same.	Must be the exact same.
Parameters	**MUST be different** (type or quantity).	**MUST be identical.**
Return Type	Can be different.	Must be the same (or covariant).
Use Case	`add(int, int)` vs `add(double, double)`	Replacing parent's `draw()` with specific child logic.

---

6. Pillar 4: Abstraction

Abstraction is the process of hiding the complex implementation details and showing only the essential features to the user. It is implemented using Abstract Classes and Interfaces.

IT STUDY HUB

Concept: Abstract Classes and Methods

An incomplete blueprint. It forces child classes to finish writing the code.

> ***Real-Life Analogy:***
> *A company boss issues a mandate: 'Every employee MUST wear a uniform.' But the boss doesn't specify what it looks like. The Security Guard and the Chef must figure out their specific uniforms themselves. The boss provided an 'Abstract' rule.*

> **Simple Explanation:**
> An `abstract class` cannot be built using `new`. It is meant strictly to be inherited. An `abstract method` is a method with NO body (no curly braces). It forces the child class to `@Override` it and provide the body.

Syntax / Structure:

```
abstract class Parent {
    public abstract void doWork(); // No {} body!
}
```

Example Code:

```
// Abstract Class (Cannot create objects of this!)
abstract class Shape {
    String color;

    // Abstract Method (Forces children to write it)
    public abstract void calculateArea();

    // Regular Method (Inherited normally)
    public void setColor(String c) {
        this.color = c;
        System.out.println("Color set to " + c);
    }
}

class Rectangle extends Shape {
    @Override
    public void calculateArea() {
        System.out.println("Calculating Rectangle Area: length * width");
```

```
    }
}

public class Geometry {
    public static void main(String[] args) {
        // Shape s = new Shape(); // ERROR! Cannot instantiate Abstract Class!

        Rectangle rect = new Rectangle();
        rect.setColor("Blue"); // Using inherited concrete method
        rect.calculateArea();  // Using overridden abstract method
    }
}
```

```
Expected Output:
Color set to Blue
Calculating Rectangle Area: length * width
```

Code Explanation:
`Shape` is an abstract concept. It doesn't make sense to 'calculate the area of a generic Shape' because we don't know the formula. By making it `abstract`, we enforce a strict rule: any class that extends `Shape` MUST mathematically provide its own `calculateArea()` logic or it won't compile.

Real-world Use:
Payment Processing. `PaymentGateway` is an abstract class with an abstract method `processPayment()`. PayPal and Stripe are child classes that implement the specific complex networking code.

Common Mistakes:

- Trying to use the `new` keyword on an abstract class.
- Writing an abstract method but forgetting to put the `abstract` keyword on the Class itself. (If a class has even one abstract method, the whole class must be abstract).

Best Practices:

- Use Abstract Classes when you have a core blueprint that shares a lot of standard data/methods, but leaves 1 or 2 specific behaviors up to the child classes to figure out.

Practice Task:
Create an abstract class `Vehicle` with abstract method `fuelUp()`. Create child `GasCar`.

IT STUDY HUB

Concept: Interfaces

A 100% abstract contract. A class `implements` an interface, promising to write all its methods.

> ***Real-Life Analogy:***
> *A USB-C port is an interface. The engineers who designed USB-C didn't build the mouse, keyboard, or hard drive. They just made a strict contract: 'If you want to plug into a computer, your device MUST follow these exact data pins.'*

> **Simple Explanation:**
> While a class can only `extends` ONE parent, a class can `implements` MULTIPLE interfaces. Interfaces traditionally only contain empty abstract methods. They are pure behavioral contracts.

Syntax / Structure:

```
interface Flyable {
    void fly(); // implicitly public and abstract
}

class Bird implements Flyable {
    @Override
    public void fly() { ... }
}
```

Example Code:

```
// Interface Contract
interface Playable {
    void playVideo();
}

interface Recordable {
    void recordAudio();
}

// Class signing MULTIPLE contracts
class Smartphone implements Playable, Recordable {
```

```
    @Override
    public void playVideo() {
        System.out.println("Streaming YouTube at 1080p.");
    }

    @Override
    public void recordAudio() {
        System.out.println("Recording voice memo...");
    }
}

public class DeviceTest {
    public static void main(String[] args) {
        Smartphone myPhone = new Smartphone();
        myPhone.playVideo();
        myPhone.recordAudio();
    }
}
```

```
Expected Output:
Streaming YouTube at 1080p.
Recording voice memo...
```

Code Explanation:
`Smartphone` signs a contract with `Playable` and `Recordable`. It is legally forced by the Java compiler to `@Override` and provide the code for `playVideo()` and `recordAudio()`. It bypassed the single-inheritance rule!

> ***Real-world Use:***
> *Connecting to a database via JDBC. Java provides the `Connection` interface. Oracle and MySQL developers write the classes that `implement` that interface.*

> **Common Mistakes:**
> - Trying to write method bodies `{}` inside a standard interface (prior to Java 8 default methods).
> - Forgetting to use the `public` keyword when overriding an interface method (Interface methods are strictly public).

Best Practices:

- Name interfaces as adjectives ending in '-able' (e.g., `Runnable`, `Serializable`, `Cloneable`).

Practice Task:
Create an interface `Swimmable` with method `swim()`. Create a `Fish` class that implements it.

Abstract Class vs Interface

Feature	Abstract Class	Interface
Inheritance Limit	A class can extend only ONE abstract class.	A class can implement MULTIPLE interfaces.
Data Fields	Can have normal state fields (String name, int age).	Only `public static final` constants allowed.
Methods	Can have a mix of regular and abstract methods.	100% abstract methods (mostly).
Core Purpose	Defines "What an object IS" (A Dog IS an Animal).	Defines "What an object CAN DO" (A Bird CAN Fly).

---

7. Access Modifiers and the 'final' keyword

Access modifiers define the visibility and security level of classes, fields, and methods across different files and packages in your application.

Modifier	Same Class?	Same Package?	Subclass (Child)?	Global (Anywhere)?
public	Yes	Yes	Yes	Yes (Open to all)
protected	Yes	Yes	Yes	No
default (no keyword)	Yes	Yes	No	No
private	Yes	No	No	No (Strictly hidden)

Rule of Thumb: Make all fields `private`. Make all methods intended for outside use `public`. Use `protected` when you want child classes to have special access to parent tools.

IT STUDY HUB

Concept: The 'final' Keyword

Locks variables, methods, and classes to prevent modification.

> ***Real-Life Analogy:***
> *Putting an item in a glass case. You can look at it, but you can't alter it, override it, or extend it.*

> **Simple Explanation:**
> When applied to a Variable: It becomes a Constant (cannot change value). When applied to a Method: It cannot be `@Override`n by a child class. When applied to a Class: It cannot be inherited (nobody can `extends` it).

Syntax / Structure:

```
final double PI = 3.14;
public final void secureMethod() { ... }
public final class SecureClass { ... }
```

Example Code:

```
final class SecuritySystem {
    public void authenticate() {
        System.out.println("Processing secure login...");
    }
}

// class HackSystem extends SecuritySystem { ... }
// ERROR: Cannot inherit from a final class!

public class FinalDemo {
    public static void main(String[] args) {
        SecuritySystem sys = new SecuritySystem();
        sys.authenticate();
    }
}
```

Expected Output:

```
Processing secure login...
```

Code Explanation:
Because `SecuritySystem` is marked `final`, no hacker can write a class that inherits from it in an attempt to override and bypass the `authenticate()` method.

Real-world Use:
The Java `String` class is natively marked `final`! This ensures nobody can maliciously override how Java fundamentally handles text in memory.

Common Mistakes:

- Trying to reassign a final variable after it has already been initialized in the constructor.

Best Practices:

- Mark variables as `final` if they are configuration values that should never change during runtime.

IT STUDY HUB

8. Project: Bank Account Management System

Let's synthesize all OOP pillars. We will create a secure `BankAccount` that utilizes **Encapsulation** to protect money, **Constructors** to initialize the user, and **Inheritance** / **Polymorphism** to build a special `SavingsAccount` with bonus features.

```
import java.util.Scanner;

// 1. Parent Class with Encapsulation
class BankAccount {
    // PRIVATE fields (Data Hiding)
    private String accountHolder;
    private double balance;

    // CONSTRUCTOR
    public BankAccount(String accountHolder, double initialDeposit) {
        this.accountHolder = accountHolder;
        if (initialDeposit > 0) {
            this.balance = initialDeposit;
        } else {
            this.balance = 0;
        }
    }

    // GETTERS
    public double getBalance() { return this.balance; }
    public String getAccountHolder() { return this.accountHolder; }

    // PUBLIC METHODS (Controlled access)
    public void deposit(double amount) {
        if (amount > 0) {
            this.balance += amount;
            System.out.println("Deposited $" + amount + ". New Balance: $" + this.balanc
        } else {
            System.out.println("Invalid deposit amount.");
        }
    }

    public void withdraw(double amount) {
        if (amount > 0 && amount <= this.balance) {
```

```
            this.balance -= amount;
            System.out.println("Withdrew $" + amount + ". New Balance: $" + this.balance
        } else {
            System.out.println("Insufficient funds or invalid amount.");
        }
    }

    // Method to be overridden
    public void displayAccountType() {
        System.out.println("Type: Standard Checking Account");
    }
}

// 2. Child Class using Inheritance and Polymorphism
class SavingsAccount extends BankAccount {
    private double interestRate;

    public SavingsAccount(String accountHolder, double initialDeposit, double rate) {
        super(accountHolder, initialDeposit); // Send data to Parent
        this.interestRate = rate;
    }

    public void applyInterest() {
        // Must use getter because balance is PRIVATE in parent!
        double interest = getBalance() * (interestRate / 100);
        deposit(interest); // Inherited method
        System.out.println("Interest applied at " + interestRate + "%.");
    }

    // Polymorphism: Method Overriding
    @Override
    public void displayAccountType() {
        System.out.println("Type: Premium Savings Account");
    }
}

// 3. Execution Logic
public class BankSystem {
    public static void main(String[] args) {
        System.out.println("=== IT STUDY HUB BANK ===");

        // Instantiate Object
        SavingsAccount myAcct = new SavingsAccount("Alex Mercer", 1000.0, 5.0);
```

```
        System.out.println("Welcome, " + myAcct.getAccountHolder());
        myAcct.displayAccountType(); // Triggers overridden version

        // Perform Transactions
        myAcct.deposit(200);
        myAcct.withdraw(50);
        myAcct.applyInterest();

        // myAcct.balance = 5000000; // COMPILER ERROR! Data is secure!

        System.out.println("Final Verified Balance: $" + myAcct.getBalance());
        System.out.println("========================");
    }
}
```

```
=== IT STUDY HUB BANK ===
Welcome, Alex Mercer
Type: Premium Savings Account
Deposited $200.0. New Balance: $1200.0
Withdrew $50.0. New Balance: $1150.0
Deposited $57.5. New Balance: $1207.5
Interest applied at 5.0%.
Final Verified Balance: $1207.5
========================
```

Step-by-Step Execution:

1. **Blueprint Definition:** We defined the `BankAccount` blueprint. We hid `balance` using `private`.
2. **Constructor Setup:** We used a Parameterized Constructor to instantly set up Alex's account with $1000.
3. **Inheritance:** `SavingsAccount` extends the bank account. It inherits the deposit and withdraw logic, so we didn't have to rewrite it! It uses `super()` to pass the name and money up to the parent.
4. **Polymorphism:** `SavingsAccount` overrides `displayAccountType()` to provide a specialized message.

5. **Encapsulation in action:** When calculating interest, the child class CANNOT access `balance` directly. It must use the parent's `getBalance()` and `deposit()` public tools to manipulate the money safely.

IT STUDY HUB

9. Module Assessment and Interview Prep

Test your conceptual understanding of Object-Oriented Programming.

A. Practice Questions (Test Your Logic)

1. Design a `Car` class with private fields for `brand` and `speed`. Write public getters and setters for both.
2. Write a default constructor for the `Car` class that sets the brand to "Unknown" and speed to 0.
3. Create a class `Animal` with a method `makeSound()`. Create a class `Cow` that extends `Animal` and overrides `makeSound()` to print "Moo".
4. What happens if you remove the `this.` keyword from a constructor parameter assignment? (e.g., `name = name;`)
5. Create an `abstract` class `Appliance` with an abstract method `turnOn()`. Create a `WashingMachine` class that extends it and provides the implementation.
6. Why can a class `implements` multiple interfaces but only `extends` one parent class?
7. Write an interface `Chargeable` with a method `chargeBattery()`. Have a `Laptop` class implement it.
8. Write a Java program to demonstrate Method Overloading by creating three `calculateArea()` methods for a Square, Rectangle, and Circle.
9. In our Bank project, why did `applyInterest()` have to use `getBalance()` instead of just typing `balance`?
10. What keyword is used to stop a class from being inherited by any other class?
11. Explain the difference between a Class and an Object in one sentence.
12. If a Parent class has a Parameterized Constructor, what MUST the Child class include in its constructor?
13. Create a class `MathUtils` with a `public static` method `add(int a, int b)`. Call it from main without using the `new` keyword.
14. What does the `protected` access modifier do that `default` does not?
15. If you instantiate two objects of the same class, do they share the same memory location for their variables?

IT STUDY HUB

B. University Viva Questions & Answers

Q: What are the four main pillars of Object-Oriented Programming?

Ans: Encapsulation, Inheritance, Polymorphism, and Abstraction.

Q: What is an Object?

Ans: An object is an instance of a class. It is a physical entity in memory that possesses state (fields) and behavior (methods).

Q: What is a Class?

Ans: A class is a logical blueprint or template used to define the properties and methods that objects of that type will share.

Q: What is a Constructor?

Ans: A special block of code initialized with the exact name of the class and no return type. It is automatically invoked when an object is created using the 'new' keyword to initialize data.

Q: Can a Constructor return a value?

Ans: No. A constructor cannot have a return type, not even 'void'. If you add 'void', it becomes a regular method.

Q: What is the 'this' keyword?

Ans: It is a reference variable that refers to the current object. It is mainly used to resolve variable shadowing between class fields and constructor parameters.

Q: What is Encapsulation?

Ans: The mechanism of wrapping data (variables) and code acting on the data (methods) together as a single unit, and hiding the data from external modification using 'private' modifiers.

Q: What is Inheritance?

Ans: A process where one class (child) acquires the fields and methods of another class (parent) using the 'extends' keyword, promoting code reusability.

Q: Does Java support multiple inheritance with classes?

Ans: No, Java does not support multiple inheritance with classes to avoid ambiguity (The Diamond Problem). However, a class can implement multiple Interfaces.

IT STUDY HUB

Q: What is the 'super' keyword?

Ans: It is a reference variable used to refer to the immediate parent class object. It is used to call parent constructors or overridden parent methods.

Q: What is Polymorphism?

Ans: The ability of a single action or method to behave differently in different contexts. It translates to 'many forms'.

Q: Explain Compile-time vs Runtime Polymorphism.

Ans: Compile-time is achieved via Method Overloading (same method name, different parameters in the same class). Runtime is achieved via Method Overriding (child class replaces parent's method).

Q: What is Abstraction?

Ans: The concept of hiding complex internal implementation details and showing only the necessary functionality to the user, achieved via abstract classes and interfaces.

Q: Can we instantiate an Abstract Class?

Ans: No. You cannot use the 'new' keyword on an abstract class. You must create a child class that extends it, provide the missing method bodies, and instantiate the child.

Q: What is an Interface?

Ans: A completely abstract blueprint containing only abstract methods (prior to Java 8) and static final constants. Classes 'implement' interfaces, signing a contract to provide the code for those methods.

Q: What is the 'final' keyword?

Ans: A modifier that locks an entity. A final variable's value cannot be changed. A final method cannot be overridden. A final class cannot be inherited.

Q: Explain the 'public' access modifier.

Ans: The code is accessible from everywhere: any class, any package, across the entire project.

Q: Explain the 'private' access modifier.

Ans: The code is strictly accessible only within the specific class where it is declared. It is entirely hidden from the outside world.

IT STUDY HUB

Q: What is the default access modifier?

Ans: If no keyword is written, it gets package-private access. It is accessible to any class within the same package, but hidden from classes in other packages.

Q: Why is the main method static?

Ans: So the JVM can invoke it directly from the class blueprint to start the program without needing to instantiate an object of the Main class first.

C. Beginner/Intermediate Interview Questions

Interview Q: Why is Encapsulation considered critical for building secure enterprise applications?

Ans: Without encapsulation, external code can directly modify sensitive fields (e.g., changing an Account Balance to a negative number). Encapsulation forces all modifications through public Setters, allowing developers to inject security checks, validation logic, and logging before allowing the memory to be altered.

Interview Q: Explain the concept of 'Variable Shadowing' in the context of constructors.

Ans: Shadowing occurs when a parameter passed into a constructor has the exact same name as a class-level field. The local parameter 'shadows' the class field, making the class field temporarily invisible. We resolve this by using 'this.fieldName' to explicitly target the class-level storage.

Interview Q: What happens if you don't write any constructor in your class?

Ans: The Java compiler automatically injects an invisible 'Default Constructor' (an empty constructor with no parameters) into the bytecode. This guarantees that object instantiation via 'new ClassName()' will always work. However, if you write ANY custom constructor, the compiler does not inject the default one.

Interview Q: Why does Java prohibit Multiple Inheritance with classes?

Ans: To avoid the 'Diamond Problem'. If Class C inherits from Class A and Class B, and both A and B have a method named 'print()', the compiler cannot determine which version of 'print()' Class C should inherit. Interfaces bypass this because interface methods generally have no bodies, leaving the implementation strictly up to Class C.

IT STUDY HUB

Interview Q: How does an Abstract Class differ functionally from an Interface?

Ans: An Abstract Class can hold state (normal variables) and concrete methods alongside abstract methods, making it ideal for defining a core hierarchy (e.g., Animal -> Dog). An Interface holds no state, only method signatures, making it a pure behavioral contract used to group unrelated objects (e.g., Airplane and Superman both implement 'Flyable').

Interview Q: Can we override a 'static' method?

Ans: No. Static methods belong to the Class, not the Object instance. Method overriding is strictly a runtime polymorphic feature based on object instantiation. If a child class creates a static method with the same name, it just 'hides' the parent's method, it does not override it.

Interview Q: What is the use case for the 'super()' method call?

Ans: When a child object is instantiated, the parent portion of that object must be built first. 'super()' explicitly calls the parent's constructor. It MUST be the very first line of code in the child's constructor. If omitted, the compiler automatically inserts an empty 'super()' call.

Interview Q: Can an interface implement another interface?

Ans: An interface cannot 'implement' another interface, but it can 'extend' one or more other interfaces, merging their behavioral contracts together.

Interview Q: If you mark a method as 'final', what are you telling other developers?

Ans: You are creating a strict architectural rule. You are telling developers who extend your class: 'You may inherit and use this method, but you are mathematically prohibited from overriding it or altering its core logic.' This is heavily used in security frameworks.

Interview Q: Is it possible to overload the 'main' method in Java?

Ans: Yes, absolutely. You can write public static void main(int x) or main(String s). However, the JVM will ONLY ever use the specific signature 'public static void main(String[] args)' as the entry point to launch the application. The overloaded ones act as normal methods.

IT STUDY HUB

10. Module Summary and Key Terms

Key Terms Glossary

- **Instantiate:** The act of creating a physical Object in memory from a Class blueprint using `new`.
- **State & Behavior:** State refers to data (fields/variables). Behavior refers to actions (methods).
- **Polymorphism:** Translates to "many forms". The ability for methods to execute different behaviors dynamically.
- **Getter / Setter:** Public methods acting as safe gatekeepers to read or edit private encapsulated data.
- **Contract:** Used to describe Interfaces, which force implementing classes to adhere to strict behavioral rules.

Module Summary

In this module, you crossed the threshold from procedural scripting into professional software architecture. You learned that OOP is about modeling the real world by creating blueprints (**Classes**) and spawning active entities in memory (**Objects**) using **Constructors**.

We explored the four pillars of OOP. You secured data behind a vault using **Encapsulation**. You eliminated redundant code by passing traits from parents to children via **Inheritance**. You modified object behaviors dynamically through **Polymorphism** (overloading and overriding). Finally, you established strict architectural rules using **Abstraction** via abstract classes and interfaces.

You applied all these concepts in the Bank Account project, proving you can architect a secure, scalable, and professional Java application.

WHAT COMES NEXT?

Module 6 — Exception Handling & File Handling

Your OOP code is elegant, but real-world users do unpredictable things. What if they enter text when your program expects a number? What if the database vanishes? In the next volume, we will tackle **Exceptions**—preventing your software from crashing when things go wrong using `try-catch` blocks. We will also break out of the console and learn to permanently save and read data using **File Handling**.

IT STUDY HUB SERIES